

Mongoose with MongoDB and Express - Full Guide

■ What is Mongoose for MongoDB?

Mongoose is an Object Data Modeling (ODM) library for MongoDB in Node.js.

It provides:

- Schema definitions (to enforce structure on MongoDB's flexible documents)
- Validation (required fields, data type checks, etc.)
- Helper methods for easier queries
- Middleware/hooks (like pre-save, post-remove)
- Relationship management (populate, refs)

■ Example without Mongoose (using MongoDB native driver)

```
const { MongoClient } = require("mongodb");

async function main() {
  const client = await MongoClient.connect("mongodb://localhost:27017");
  const db = client.db("school");
  const students = db.collection("students");

  await students.insertOne({ name: "Alice", age: 20 });
  console.log(await students.find().toArray());

  client.close();
}

main();
```

■ Example with Mongoose

```
const mongoose = require("mongoose");

// Connect
mongoose.connect("mongodb://localhost:27017/school");

// Define schema
const studentSchema = new mongoose.Schema({
  name: { type: String, required: true },
  age: Number,
  enrolled: { type: Boolean, default: true }
});

// Create model
const Student = mongoose.model("Student", studentSchema);

// Use model
async function run() {
  const s1 = new Student({ name: "Alice", age: 20 });
  await s1.save();
  console.log(await Student.find());
}

run();
```

■ Express + Mongoose Sample App (Student API)

```

const express = require("express");
const mongoose = require("mongoose");

const app = express();
app.use(express.json()); // to parse JSON body

// 1. Connect to MongoDB
mongoose
  .connect("mongodb://127.0.0.1:27017/schoolDB")
  .then(() => console.log("■ MongoDB connected"))
  .catch((err) => console.error("■ Connection error:", err));

// 2. Define Schema
const studentSchema = new mongoose.Schema({
  name: { type: String, required: true },
  age: Number,
  email: { type: String, unique: true },
});

// 3. Create Model
const Student = mongoose.model("Student", studentSchema);

// 4. Routes

// ➡■ Create a student
app.post("/students", async (req, res) => {
  try {
    const student = new Student(req.body);
    await student.save();
    res.status(201).json(student);
  } catch (err) {
    res.status(400).json({ error: err.message });
  }
});

// ➡■ Get all students
app.get("/students", async (req, res) => {
  const students = await Student.find();
  res.json(students);
});

// ➡■ Get student by ID
app.get("/students/:id", async (req, res) => {
  try {
    const student = await Student.findById(req.params.id);
    if (!student) return res.status(404).json({ error: "Not found" });
    res.json(student);
  } catch (err) {
    res.status(400).json({ error: "Invalid ID" });
  }
});

// ➡■ Update student
app.put("/students/:id", async (req, res) => {
  try {
    const student = await Student.findByIdAndUpdate(req.params.id, req.body, {
      new: true,
      runValidators: true,
    });
    if (!student) return res.status(404).json({ error: "Not found" });
    res.json(student);
  } catch (err) {
    res.status(400).json({ error: err.message });
  }
});

// ➡■ Delete student
app.delete("/students/:id", async (req, res) => {
  try {
    const student = await Student.findByIdAndDelete(req.params.id);
    if (!student) return res.status(404).json({ error: "Not found" });
    res.json({ message: "Deleted successfully" });
  }
});

```

```

    } catch (err) {
      res.status(400).json({ error: "Invalid ID" });
    }
  });

// 5. Start Server
const PORT = 8000;
app.listen(PORT, () => {
  console.log(`■ Server running at http://localhost:${PORT}`);
});

```

■ Simple CRUD App using Express + Mongoose

```

const express = require("express");
const mongoose = require("mongoose");

const app = express();
app.use(express.json());

// ■ Connect to MongoDB
mongoose.connect("mongodb://127.0.0.1:27017/testdb")
  .then(() => console.log("MongoDB connected"))
  .catch(err => console.error(err));

// ■ Schema & Model
const userSchema = new mongoose.Schema({
  name: String,
  email: String
});
const User = mongoose.model("User", userSchema);

// ■ Routes

// Create
app.post("/users", async (req, res) => {
  const user = new User(req.body);
  await user.save();
  res.send(user);
});

// Read all
app.get("/users", async (req, res) => {
  const users = await User.find();
  res.send(users);
});

// Read by ID
app.get("/users/:id", async (req, res) => {
  const user = await User.findById(req.params.id);
  res.send(user);
});

// Update
app.put("/users/:id", async (req, res) => {
  const user = await User.findByIdAndUpdate(req.params.id, req.body, { new: true });
  res.send(user);
});

// Delete
app.delete("/users/:id", async (req, res) => {
  await User.findByIdAndDelete(req.params.id);
  res.send({ message: "User deleted" });
});

// ■ Start Server
app.listen(3000, () => console.log("Server running on http://localhost:3000"));

```

■ Test API in Postman/Thunder Client

1. POST → `http://localhost:3000/users`
`{ "name": "Alice", "email": "alice@example.com" }`
2. GET → `http://localhost:3000/users` → all users
3. GET → `http://localhost:3000/users/<id>` → one user
4. PUT → `http://localhost:3000/users/<id>` → update user
5. DELETE → `http://localhost:3000/users/<id>` → delete user